ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Organisation of Islamic Cooperation (OIC)

Department of Computer Science and Engineering (CSE)

Project Report

DORMATRIX

IUT Female Dormitory Management System

SWE 4304: Software Project Lab I

Team Members

Student ID	Team Member
230042141	Ayman Binta Altaf Nondiny
230042135	Khadiza Sultana
230042114	Procheta Silvie
230042139	Sayma Tasnim

Supervisor

Md. Tariquzzaman

Junior Lecturer
Department of Computer Science and Engineering
Islamic University of Technology

Project Title	DORMATRIX: IUT Female Dormitory Management System
Course	SWE 4304 - Software Project Lab I
Submission Date	March 17, 2026

Contents

Executive Summary 1

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 2

1.3 Motivation . . . 2

1.4 Objectives . . . 2

1.5 Scope of the System . . . 3

2 Project Overview 3

2.1 Project Identity . . . 3

2.2 Primary User Roles . . . 3

2.3 High-Level Workflow . . . 4

2.4 Major Deliverables . . . 5

3 Requirements Analysis 5

3.1 Functional Requirements . . . 5

3.2 Non-Functional Requirements . . . 6

3.3 Assumptions and Constraints . . . 7

4 System Architecture and Design 7

4.1 Architectural Pattern . . . 7

4.2 Code base and Package Distribution . . . 8

4.3 Custom-Built Libraries . . . 8

4.4 Data Storage Design . . . 9

4.5 Security and Credential Handling . . . 10

4.6 Key Algorithms and Design Logic . . . 10

4.6.1 First-Fit Allocation . . . 10

4.6.2 Complaint Policy Engine . . . 10

4.6.3 Time and Demo Management . . . 10

5 Module-Wise Implementation Details 10

5.1 Authentication Module . . . 10

5.2 Dashboard and Terminal UI Module 11
5.3 Room Management Module 12
5.4 Facility Booking Module 13
5.5 Store, Balance, and Due Management Module 14
5.6 Cafeteria and Meal Token Module 14
5.7 Complaint Management Module 15
5.8 Lost & Found Module 16
5.9 Announcement and Scheduling Module 17
5.10 Routine, Contacts, and Profile Management 17
5.11 Administrative Control Module 17

6 User Experience, Interface Design, and Visual Planning 18
6.1 Visual Identity 18
6.2 Navigation Design 18
6.3 Reserved Figure Plan for the Final Report 18

7 Implementation Evidence and Technical Notes 19
7.1 Entry Point and Runtime Setup 19
7.2 Run Environment 19
7.3 Error Handling 20
7.4 Persistence Transparency 20
7.5 Extensibility 20

8 Testing and Quality Assurance 20
8.1 Testing Scope 20
8.2 What the Test Code Covers 20
8.3 Testing Interpretation 21
8.4 Manual Validation and Demo Readiness 21
8.5 Risk Areas 21

9 Project Management and Team Contributions 22
9.1 Contribution Overview 22
9.2 Planned Timeline vs. Realized System 22
9.3 Educational Value of the Work Distribution 23

10 Limitations and Future Improvements **23**
10.1 Current Limitations 23
10.2 Future Enhancements 24

11 Personal Reflection Placeholders **24**
11.1 Procheta Silvie 24
11.2 Khadiza Sultana 24
11.3 Sayma Tasnim 25
11.4 Ayman Binta Altaf Nondiny 25

12 Conclusion **25**

Executive Summary

Dormatrix is a detailed command-line dormitory management system developed for the IUT Female Dormitory. The project brings together a wide range of dormitory services into one integrated application: authentication, room information, room change handling, complaint submission and routing, facility booking, store and due tracking, cafeteria token purchase, announcements, schedules, routines, emergency contacts, and lost & found management. Instead of treating hall services as isolated tasks handled through informal communication or paper-based follow-up, Dormatrix centralizes those services behind role-based dashboards designed for students, attendants, maintenance workers, store staff, hall office staff, administrators, and cafeteria managers.

The current codebase shows a substantial engineering effort. The source tree contains 185 Java source files organized around an MVC + Repository structure, with approximately 30,140 lines of Java source. The project also includes custom-built utility classes such as `MyArrayList`, `MyString`, `MyOptional`, a custom password hashing function, slot allocation logic, and a file-based persistence layer. Rather than depending on a database server or heavyweight framework, Dormatrix stores structured application state in plain-text files, allowing the system to remain portable, lightweight, and easy to demonstrate in a lab setting.

This report expands the initial proposal into a more complete and detailed final project document. It covers the motivation of the project, user roles, module-by-module implementation, architectural choices, technical mechanisms, testing scope, data storage design, contribution breakdown, and future directions. Reserved spaces are also included for UI screenshots, class diagrams, and each member's personal reflection so the document can be finalized further without changing the report structure.

1 Introduction

1.1 Background

Life inside a residential dormitory depends on a large number of small but important operational activities: room allocation, maintenance support, cafeteria coordination, internal announcements, store purchases, routine tracking, and emergency communication. In practice, these services are often fragmented. Some are handled verbally, some through paper notes, and others by individual staff members who maintain personal records. This fragmentation creates delay, confusion, repetition, and a lack of visibility.

Dormatrix was designed as a digital dormitory operations platform tailored to the context of the IUT Female Hall. The project focuses on giving each stakeholder a role-specific workflow while keeping the overall system unified. A student should be able to book a facility, track a complaint, view announcements, buy meal tokens, and check her dues from one application. At the same time, attendants, maintenance workers, store staff, hall office staff, and administrators should see only the tools relevant to their responsibilities.

1.2 Problem Statement

Before a centralized system exists, the following operational problems typically appear:

- Complaint handling becomes slow because submission, escalation, assignment, and status updates are not tracked in one place.
- Shared facilities such as the study room, laundry area, and fridge are difficult to use fairly without a booking and release mechanism.
- Store purchases and credit-based dues can become unstructured, making it hard to maintain balance records and payment history.
- Meal management is inconvenient when students cannot selectively purchase only the meals they need.
- Important notices about schedules, events, or administrative updates may not reach all residents consistently.
- Staff coordination becomes inefficient without a clear schedule board or task list.

Dormatrix addresses these problems by turning the hall into a structured digital workflow system rather than a collection of disconnected manual actions.

1.3 Motivation

The project motivation emerged from very practical hall-life pain points already identified in the proposal report: complaint delays, unmanaged shared resources, lack of structured store accounting, limited food flexibility, and the absence of integrated visibility across dormitory operations. The final implementation keeps those original motivations intact while pushing the system further toward usability and demonstration value through a themed command-line interface, role-aware dashboards, demo-time acceleration, and dedicated service modules.

1.4 Objectives

The main objectives of Dormatrix are:

- to design one unified dormitory management platform for multiple hall stakeholders;
- to apply object-oriented design and modular package organization in a real project setting;
- to build a file-persistent system that can run without database setup;
- to ensure that each user role receives a tailored dashboard and permission set;
- to improve visibility, accountability, and coordination for complaints, facilities, store operations, and cafeteria services;

- to produce a polished demonstration-ready system with maintainable code and supporting documentation.

1.5 Scope of the System

Dormatrix is intentionally focused on the female dormitory context. The scope includes:

- user authentication and role-based access;
- room viewing, room change applications, and room administration;
- complaint submission, prioritization, assignment, and resolution tracking;
- facility booking for laundry, study room, and fridge slots;
- cafeteria token purchase, menu viewing, token validation, and Ramadan time switching;
- departmental store inventory, shopping cart, sales, student balances, and dues;
- lost & found records;
- announcements, routines, schedules, emergency contacts, and profile editing.

The system does not attempt to solve all university-wide dormitory management challenges. It does not currently include networked multi-user synchronization, mobile apps, or relational database deployment. Those are treated as future enhancements.

2 Project Overview

2.1 Project Identity

Dormatrix is a role-aware command-line application that combines dormitory operations and visual identity. The name blends *Dormitory* and *Matrix*, reflecting both the interconnected nature of hall services and the matrix-rain aesthetic used in the UI. The system is deliberately presentation-friendly: each role has a themed dashboard, menus can be navigated with both numbers and arrow keys, and the meal system supports accelerated demo mode for live presentation scenarios.

2.2 Primary User Roles

The current implementation supports seven operational roles and an additional main entry screen theme.

Table 1: Role summary within Dormatrix

Role	Theme	Options	Primary responsibilities
Student	Navy Blue	11	Room status, booking, token purchase, store, dues, complaints, routine, announcements, contacts, profile
Attendant	Deep Teal	7	Complaint monitoring, worker scheduling, found item updates, announcement posting, emergency contacts
Maintenance Worker	Steel Blue / Gray	4	Work field awareness, task handling, visit tracking, routine visibility
Store-in-Charge	Amber / Brown	4	Inventory control, purchase processing, sales and summary reporting
Hall Office	Hot Pink / Magenta	5	Student room data, complaint visibility, worker scheduling, hall administration
Admin	Deep Red	5	Account creation/deletion, user search, room management, system control
Cafeteria Manager	Golden Yellow	5	Weekly menu updates, event schedule, token verification, Ramadan mode

2.3 High-Level Workflow

At a conceptual level, Dormatrix follows the same operational pattern across modules:

1. A user selects a role from the main screen.
2. The authentication layer validates identity and password against role-specific user files.
3. On successful login, the appropriate dashboard controller and CLI view are loaded.
4. The user performs actions within her or his role boundary.
5. Controllers apply business logic, update model objects, and persist relevant state to plain-text files.
6. Other stakeholders view the resulting data through their own dashboards, creating a connected dormitory workflow.

Integrated Dormitory Workflow Snapshot
Student actions such as complaints, bookings, purchases, and token requests generate persistent records. These records become visible to **attendants**, **maintenance workers**, **store staff**, **hall office staff**, or the **cafeteria manager**, depending on the module. This creates a closed operational loop instead of isolated requests with no traceability.

2.4 Major Deliverables

The project deliverables can be grouped into four categories:

- **Software system:** the Java implementation of Dormatrix.
- **Project documentation:** proposal report, final report, README, and supporting explanations.
- **Presentation materials:** slides prepared across presentations.
- **Testing artifacts:** JUnit-based test code and validation coverage for core logic.

3 Requirements Analysis

3.1 Functional Requirements

The functional requirements were realized as distinct but connected modules. Table 2 summarizes the main functional requirement categories.

Table 2: Functional requirement matrix

Requirement area	Main actors	Required system behavior
Authentication and access control	All users	Validate login credentials, apply role-specific routing, support masked password entry, separate user files by role
Room information and room change	Student, Hall Office, Admin	Show room status, manage room records, preview availability, submit and review room-change applications
Facility booking	Student, Attendant, Hall Office	Reserve shared resources, prevent double booking, auto-release or track facility state through persistent files

Requirement area	Main actors	Required system behavior
Complaint management	Student, Attendant, Maintenance Worker, Hall Office	Submit complaints, detect emergency terms, assign status and worker field, allow follow-up across lifecycle stages
Store and balance control	Student, Store-in-Charge	Show inventory, process purchases, maintain balances and dues, generate sales history and summaries
Cafeteria and meal token system	Student, Cafeteria Manager	Publish menu, enforce meal windows, sell meal tokens, verify tokens, support Ramadan mode
Announcements and schedules	Attendant, Hall Office, Maintenance Worker, Student	Publish notices, track worker visits and routines, make operational information visible to relevant roles
Lost & found	Student, Attendant	Report lost items, register found items, support claiming and traceability
Emergency contacts and profile management	Student, Attendant and other roles	View contact lists, manage role-relevant personal information, update profile details
Administrative control	Admin	Create/delete accounts, search records, support top-level data operations

3.2 Non-Functional Requirements

Beyond feature completion, the project also demonstrates a number of non-functional considerations:

- **Usability:** role-aware dashboards reduce clutter; arrow and numeric navigation support smooth CLI interaction.
- **Readability:** visual hierarchy, terminal color themes, bordered panels, and menu highlights improve operator confidence.
- **Portability:** the application runs without a database server and keeps persistence in text files.
- **Maintainability:** the codebase is separated into packages for CLI, controllers, models, repositories, libraries, and utilities.
- **Security:** passwords are hashed rather than stored in plain text.

- **Demonstrability:** demo mode compresses time-sensitive workflows so meal-slot transitions can be shown during presentation.

3.3 Assumptions and Constraints

The project also inherits a few constraints from its design choices:

- the system assumes text-file integrity and does not yet handle concurrent multi-user editing;
- the interface is optimized for terminals capable of ANSI true-color rendering;
- the project is structured primarily for local execution, classroom demonstration, and lab evaluation;
- file-based persistence simplifies deployment but limits large-scale growth and simultaneous access;
- some workflows are optimized for dormitory operations and would need adaptation before reuse in a different hall environment.

4 System Architecture and Design

4.1 Architectural Pattern

Dormatrix follows an MVC + Repository approach. The CLI layer acts as the view layer. Controllers implement business rules, repositories organize data access, and models represent domain entities. This structure keeps interaction logic separate from persistence logic and makes the system easier to extend module by module.

Table 3: Layered View of the System

CLI Layer (Views) Dashboards, forms, panels, views, and interactive user flows
⇓
Controller Layer (Business Logic) Authentication, room, facilities, complaint, store, food, routine, profile, scheduling
⇓
Repository / File Access Layer Structured reading and writing of plain-text records
⇓
Model Layer Users, rooms, complaints, meals, tokens, balances, dues, routines, visits, items

4.2 Code base and Package Distribution

The package distribution highlights the scope of the implementation.

Table 4: Source package distribution

Top-level source area	Interpretation
CLI	User-facing dashboards, forms, and screen-specific interaction logic
Controllers	Operational logic for authentication, rooms, food, store, complaints, scheduling, and more
Models	Domain entities and enums representing dormitory state
Repositories	Persistence-facing classes for file-backed records
Libraries	Reusable custom utilities and core data structures
Exceptions	Domain-specific error handling classes
Utilities	Shared helpers for rendering, time, input, colors, and role mapping
Tests	Broad JUnit-based validation coverage
Module wrappers	Additional complaint module service layer

4.3 Custom-Built Libraries

A notable technical aspect of Dormatrix is the deliberate use of custom supporting libraries instead of relying on standard Java collection classes for the main data structure layer. This choice strengthens the educational value of the project because the team implemented not only application workflows but also reusable low-level helpers.

Table 5: Selected custom library classes

Package	Class	Technical purpose
`libraries/collections`	`MyArrayList`	Dynamic array-backed list replacement used in place of standard list utilities
`libraries/collections`	`MyString`	Wrapper that provides split, lowercase conversion, replacement, containment checks, and safer string handling
`libraries/collections`	`MyOptional`	Optional-style wrapper for safer nullable access
`libraries/hashing`	`HashFunction`	Password hashing using a DJB2 + XOR combination approach
`libraries/slots`	`FirstFitAllocator`	First-fit allocation logic for fridge booking and other slot-driven tasks
`libraries/file`	`TextFile`	File reading and writing helper
`libraries/logs`	`Logger`	Lightweight event logging support

4.4 Data Storage Design

The persistence mechanism is intentionally straightforward and transparent. Application records are stored in structured text files grouped by domain under the `data/` directory. This design is particularly suitable for an SPL course project because records can be inspected manually, reset easily, and demonstrated without external dependencies.

Table 6: Representative persistent storage files

File path	Stored information
`data/users/students.txt`	Student identity, department, hashed password, phone, email, and room assignment
`data/users/admin.txt`	Administrative account credentials
`data/users/hall_attendants.txt`	Hall attendant records
`data/users/maintenance_workers.txt`	Maintenance workers with field specialization
`data/users/store_in_charges.txt`	Store staff records
`data/users/hall_officers.txt`	Hall office staff records
`data/users/cafeteria_managers.txt`	Cafeteria manager records
`data/rooms/rooms.txt`	Room identifiers, capacities, and occupancy data
`data/rooms/room_change_applications.txt`	Pending and processed room change applications
`data/complaints/complaints.txt`	Complaint record stream including status and priority
`data/facility/laundrySlots.txt`	Timed laundry-slot occupancy
`data/facility/studyRoomSlots.txt`	Study-room seat booking state
`data/foods/config.txt`	Ramadan mode toggle persisted across runs
`data/foods/tokens.txt`	Meal token records and statuses
`data/foods/weekly_menu.txt`	Weekly cafeteria menu entries
`data/inventories/inventory.txt`	Store inventory records
`data/inventories/balances.txt`	Student balance file for stored-value or cash tracking
`data/store/dues.txt`	Outstanding student due ledger
`data/store/sales.txt`	Sales history
`data/routines/student_routines.txt`	Routine entries per student
`data/schedules/worker_visits.txt`	Maintenance visit scheduling records
`data/announcements/announcements.txt`	Hall notices and announcement records
`data/contacts/emergency_contacts.txt`	Emergency contact entries
`data/lostItems.txt`	Lost-item reports
`data/foundItems.txt`	Found-item records

4.5 Security and Credential Handling

The authentication flow reads role-specific user files and validates the given password after hashing it using the custom `HashFunction` implementation. The provided source indicates that hashing combines DJB2-style rolling updates with XOR-based mixing before producing a positive hexadecimal representation. For a course project, this is a meaningful improvement over plain-text password storage and demonstrates that security considerations were treated as part of the system design rather than an afterthought.

4.6 Key Algorithms and Design Logic

4.6.1 First-Fit Allocation

The `FirstFitAllocator` scans slot arrays sequentially and returns the first available position. This is especially useful for fridge slot allocation because it provides deterministic, simple, and efficient assignment without maintaining a more complex placement model.

4.6.2 Complaint Policy Engine

The `ComplaintPolicy` class uses complaint category plus emergency-keyword analysis to determine whether a complaint should be elevated. Terms such as *fire*, *smoke*, *sparks*, *electric shock*, *flood*, or *burst pipe* trigger emergency behavior. The policy also maps categories to recommended worker fields such as electrician, plumber, internet technician, or cleaning staff. This gives the complaint module a meaningful decision layer instead of merely storing user-submitted text.

4.6.3 Time and Demo Management

The `TimeManager` supports:

- normal real-time execution,
- demo mode, where 20 real minutes simulate a full 15-hour day from 07:00 to 22:00,
- Ramadan mode, which swaps meal windows for Suhoor, Iftar, and dinner timing.

This is a particularly smart design choice because a live evaluator can observe multiple food-system states during one short demonstration session.

5 Module-Wise Implementation Details

5.1 Authentication Module

Authentication is one of the foundational modules and is central to the user experience. The `AuthController` normalizes role names, selects the correct role-based file, reads user records, hashes the input password, and compares it to the stored hash. The module

also supports masked password input through JLine when the terminal environment is capable of advanced console handling.

Implementation significance:

- separates users by role rather than maintaining one monolithic user table;
- uses a role-normalization strategy to tolerate naming variation;
- avoids plain-text password storage;
- supports CLI-friendly password masking and cancellation handling.

Why this matters in the overall system: every other module depends on correct authentication because the dashboard loaded after login determines what a user can see and do.

Authentication and Login Interface

Reserve this space for the final login screenshot showing user ID input, masked password entry, and role-aware theme transition.

Figure 1: Authentication and Login Interface

5.2 Dashboard and Terminal UI Module

One of Dormatrix's strongest identities lies in its terminal presentation. The system is not a plain menu list; it uses a visual layer based on ANSI true-color rendering, bordered panels, active theme switching, dynamic alignment, and special effects such as matrix-rain animation on first login.

The UI layer includes:

- a role selection entry screen;
- dashboard layouts for all supported roles;
- themed input prompts and bordered components;
- gradient rendering and background fill helpers;
- arrow-key as well as number-based menu navigation.

This module matters because it turns a technically dense project into a polished, presentation-ready system. It also reflects direct ownership by members who worked on UI and dashboard behavior.

Main Role Selection and Dashboard Theme View

Reserve this space for a screenshot that captures the role selection interface and the visual distinction between dashboards.

Figure 2: Main Role Selection and Dashboard Theme View

5.3 Room Management Module

Room management covers both information visibility and administrative change handling. Students can view their room status, while hall office personnel and administrators can update room-related data. The source tree also includes room change request handling, room availability preview dashboards, and room service/controller separation.

Implemented concerns include:

- current room assignment visibility;
- unassigned-state support for new students;
- room capacity and occupancy tracking;
- room-change applications with persistent record storage;
- dashboard flows for both student-side request submission and administrative-side review.

This module contributes heavily to the "dormitory management" identity of the system because room visibility is one of the most essential resident services.

Room Status and Room Change Workflow

Reserve this space for one or two screenshots showing student room information and the room change request dashboard or form.

Figure 3: Room Status and Room Change Workflow

5.4 Facility Booking Module

The facility booking module addresses a concrete daily-life problem inside the dormitory. Instead of informal reservation behavior, Dormatrix formalizes shared resource booking through structured controllers and stored booking data.

The current facility implementation spans three main services:

1. **Laundry booking:** students can reserve from available machines, and slot persistence is maintained in file storage.
2. **Study room booking:** seat-based occupancy is tracked to prevent unfair reservation behavior.
3. **Fridge slot allocation:** first-fit placement is used to assign the next available slot.

This module is a strong example of how the project combines algorithmic logic, real dormitory use cases, and persistent state management.

Facility Booking Screens

Reserve this space for screenshots of laundry slot selection, study-room seat allocation, and fridge slot placement.

Figure 4: Facility Booking Screens

5.5 Store, Balance, and Due Management Module

The store system is one of the most substantial modules in the codebase. It combines inventory handling, shopping-cart-style purchase flow, sales history, due recording, and balance tracking. This aligns directly with the division of work where Procheta handled the store module and balance-related functionality.

Core responsibilities include:

- maintaining inventory items and quantities;
- allowing students to browse products and build a cart;
- processing purchases either from balance or on credit;
- updating due ledgers when credit is used;
- generating sales records and summaries for store staff;
- providing account-style visibility of student purchases and dues.

This module is operationally important because dormitory stores often run on partial trust and informal records. Dormatrix turns those records into an explicit ledger with traceable purchase history.

Operational Value of the Store Module
The store subsystem does not only record items sold. It creates a financial trail through balances, dues, and sales summaries. That makes it useful to three groups at once: students who need visibility, store staff who need accountability, and hall management who may later need audit-ready summaries.

Store Inventory, Cart, and Ledger Views

Reserve this space for screenshots of the inventory list, shopping cart confirmation screen, and store ledger or sales summary.

Figure 5: Store Inventory, Cart, and Ledger Views

5.6 Cafeteria and Meal Token Module

The cafeteria module introduces a time-sensitive workflow that goes beyond static menus. Students can purchase meal tokens only within valid windows, token status is tracked,

and the cafeteria manager can update menus or toggle Ramadan mode. This reflects Khadiza's ownership of the cafeteria module.

Key implementation ideas include:

- time-aware purchase restriction;
- distinct meal windows for breakfast, lunch, and dinner;
- alternate Ramadan schedule with Suhoor and Iftar support;
- token states such as active, used, or expired;
- manager-side verification and menu maintenance.

This is one of the most demonstration-friendly modules because its correctness depends on time logic rather than static CRUD alone. The use of demo mode makes the module much easier to showcase.

Meal Token Purchase and Verification Screens

Reserve this space for screenshots showing token purchase, menu visibility, and cafeteria manager token verification.

Figure 6: Meal Token Purchase and Verification Screens

5.7 Complaint Management Module

Complaint management is a central service feature and directly reflects Nondiny's ownership of the complaint module. The workflow is more sophisticated than simple submission because the module models category, description, priority, emergency detection, assignment, and status progression.

Core complaint lifecycle:

```
SUBMITTED ⟶ ASSIGNED ⟶ IN_PROGRESS ⟶ RESOLVED
```

Why this module matters:

- it captures a major real-life dormitory pain point;
- it automates prioritization through the complaint policy engine;

- it enables staff coordination by connecting students, attendants, and maintenance workers;
- it improves accountability by keeping a traceable complaint history.

The complaint engine also demonstrates how the project moves from plain record storage toward rule-based decision making. A complaint about smoke, sparks, burst pipe, or electric shock is not treated as a normal request; it is elevated appropriately.

Complaint Submission and Tracking Views

Reserve this space for screenshots of complaint submission, attendant complaint handling, and worker task list screens.

Figure 7: Complaint Submission and Tracking Views

5.8 Lost & Found Module

The lost & found module is a comparatively smaller feature, but it fills a realistic service gap inside dorm life. Students can report missing items, attendants can register found items, and records can be claimed or updated as statuses change. Sayma's ownership of the facility module and lost & found is directly visible here.

This module improves day-to-day usability because dormitory residents frequently misplace objects, and a searchable, timestamped record is far more effective than ad-hoc verbal coordination.

Lost & Found Interface

Reserve this space for screenshots of item reporting, found item registration, and claim status display.

Figure 8: Lost & Found Interface

5.9 Announcement and Scheduling Module

Announcements and scheduling form another important coordination layer within the system. This area maps to Nondiny's work on announcements and scheduling. The module enables hall-wide notice distribution and staff coordination rather than relying on word-of-mouth communication alone.

Functional role of the announcement and schedule subsystem:

- attendants can publish hall notices;
- students can read the most recent notices;
- worker visit entries can be tracked;
- attendants and hall office staff can monitor operational schedules.

This module strengthens the project because it expands Dormatrix from a transaction system into an information distribution platform.

Announcement Board and Worker Schedule Views

Reserve this space for screenshots of the announcement board, schedule board, and worker visit tracking screens.

Figure 9: Announcement Board and Worker Schedule Views

5.10 Routine, Contacts, and Profile Management

Although these may appear auxiliary compared to complaints or store operations, they make the project feel complete. Students can maintain routines, attendants can view them, users can manage profiles, and emergency contacts provide a safety-oriented support feature.

These modules help the application feel like a comprehensive dormitory platform instead of a disconnected collection of independent mini-features.

5.11 Administrative Control Module

The administrator dashboard offers top-level account and room control. The codebase includes account creation, deletion, record search, and room management support. This module is structurally important because it provides maintainability and lifecycle control for the rest of the system.

6 User Experience, Interface Design, and Visual Planning

6.1 Visual Identity

Dormatrix intentionally uses a memorable visual style rather than a bare terminal menu. The design decisions visible in the source and README include:

- ANSI true-color backgrounds and text accents;
- separate theme palettes per role;
- a matrix-rain animation that creates a recognizable product identity;
- bordered dashboards and summary panels;
- active selection highlighting for menu traversal;
- adaptable spacing based on terminal size.

This is especially valuable for an SPL project because presentation quality strongly affects how the software is perceived during evaluation.

6.2 Navigation Design

The interface supports both number-based and arrow-based selection, which reduces friction for first-time users. Number entry is efficient for experienced users, while arrow navigation makes the system more discoverable during live demonstrations.

6.3 Reserved Figure Plan for the Final Report

The final printed report will benefit significantly from selected UI screenshots. The reserved placeholders inserted throughout this report are intended to be replaced with high-resolution captures from the actual demo environment. The recommended screenshots are:

- main role selection screen;
- login page with masked password prompt;
- student dashboard;
- complaint workflow screens;
- store inventory and ledger screens;
- cafeteria token purchase and verification screens;
- room status and room change screens;
- announcement board and schedule views;
- lost & found pages;

- admin account management screen.

Student Dashboard Overview

Reserve this space for a polished screenshot of the full student dashboard with theme colors and option list visible.

Figure 10: Student Dashboard Overview

Admin and Hall Office Dashboards

Reserve this space for screenshots showing the administrative view and the hall office dashboard, especially room and account management areas.

Figure 11: Admin and Hall Office Dashboards

7 Implementation Evidence and Technical Notes

7.1 Entry Point and Runtime Setup

The main class initializes time management and starts in demo mode before launching the main dashboard. This is a small but meaningful design choice: it proves that presentation readiness was considered from the application's entry point itself.

7.2 Run Environment

The project is packaged to run through Windows batch scripts and recommends a terminal that supports ANSI true-color output such as WezTerm. This matches the system's emphasis on visual terminal rendering. It also means the project is not merely command-line in the minimal sense; it uses terminal capabilities as part of the interface design.

7.3 Error Handling

The codebase includes dedicated exception classes such as invalid input exceptions, slot-unavailable exceptions, inventory exceptions, and configuration load exceptions. This suggests the team intentionally separated normal control flow from exceptional conditions.

7.4 Persistence Transparency

Because plain-text files are used throughout the system, the project remains very transparent for debugging and review. For academic assessment, this is useful because evaluators can inspect raw records directly and trace how user actions affect persistent state.

7.5 Extensibility

The package structure leaves clear room for extension. Potential future additions such as payment integration, richer search, notification layers, or database migration could be added without rewriting the whole application. The existence of separate repositories, controllers, and domain models helps preserve that extensibility.

8 Testing and Quality Assurance

8.1 Testing Scope

Testing is a substantial part of the current source base and is explicitly associated with Khadiza's work and partial testing support from Nondiny. The provided test directory contains two large JUnit-based suites: `UnitTests.java` and `TerminalUITest.java`. Together, the source includes **409 annotated test methods**, which is a strong signal of deliberate validation effort.

8.2 What the Test Code Covers

From the structure of the test imports and method names, the suite covers a broad range of components, including:

- custom collection and string utilities;
- hashing behavior;
- slot allocation logic;
- validation exceptions for account data;
- role mapping and profile control;
- room and room-change workflows;
- routine and schedule controllers;
- inventory, sales, dues, and purchase handling;

- lost & found processing;
- terminal UI helper methods such as padding, gradient rendering, and theme application.

8.3 Testing Interpretation

For a project of this size, the test suite is important for two reasons. First, it shows that the team did not stop at feature construction; they also attempted to validate logic, formatting helpers, and persistence-related behaviors. Second, it increases confidence in sensitive workflows such as account creation, file-backed controllers, and custom library behavior.

Important Note for the Final Viva or Written Submission

This report intentionally states that the repository *contains* 409 test methods. It does not claim a specific real-time pass rate here because that depends on the exact runtime environment and the most recent code revision. During final submission, this section can be extended with an executed test summary if desired.

8.4 Manual Validation and Demo Readiness

In addition to automated tests, Dormatrix also relies on strong manual demo validation. This is particularly relevant for:

- terminal layout correctness,
- color themes and animation behavior,
- navigation smoothness,
- real-time or demo-time meal window transitions,
- integrated multi-role workflow demonstration.

8.5 Risk Areas

The major areas that deserve continuous validation are:

- file consistency after repeated writes;
- edge cases in role-based routing;
- synchronization assumptions if multiple users ever interact with the same data files;
- UI behavior under smaller terminal dimensions;
- long-term maintainability of text-file formats as the schema evolves.

9 Project Management and Team Contributions

9.1 Contribution Overview

The module ownership information provided for this report is summarized below. This section should be preserved as the definitive contribution record unless the team later decides to refine wording.

Table 7: Team contribution matrix

Team member	Major contribution areas
Procheta Silvie	Store module, balance-related functionality
Sayma Tasnim	Facility module and lost & found module
Khadiza Sultana	Authentication, UI, cafeteria module, and testing
Ayman Binta Altaf Nondiny	Complaint module, announcement and scheduling module, testing, and UI

9.2 Planned Timeline vs. Realized System

The proposal outlined a staged development plan beginning with dashboard foundations, then authentication, room and complaint modules, facility booking, store and food systems, lost & found, announcements, testing, and final documentation. The current codebase aligns strongly with that roadmap. The implementation now includes each of those planned areas in code form, along with additional visual polish and richer testing evidence.

Table 8: Progress against proposed phases

Planned phase	Intended output	Observed implementation status
Dashboard foundation	Role-based dashboards and navigation skeleton	Implemented with themed dashboards and CLI menus
Authentication and user system	Login, role routing, core user records	Implemented with hashed passwords and role-based files
Room and complaint system	Room info and complaint flow	Implemented with room services and complaint lifecycle handling
Facility booking	Study room, fridge, laundry	Implemented with separate controllers and slot logic
Store and food system	Inventory, dues, tokens, cafeteria	Implemented with controllers, files, balances, dues, and token logic
Lost & found, announcements, events	Hall communication and item tracking	Implemented as dedicated modules
Backend enhancement	File handling, organization, exceptions	Visible through package separation and specialized exceptions
Testing and debugging	Unit tests	Strong evidence present in the test source
Documentation and presentation	Reports, README, slides, demo assets	Proposal, README, and presentation support are present; this final report extends the set

9.3 Educational Value of the Work Distribution

The team's distribution is strong from a software project perspective because it mixes domain modules (store, cafeteria, complaints, facilities) with cross-cutting work (UI, testing, presentation, documentation). This reflects how real projects work: not every contribution is only about one controller or one model. Some contributions improve the product through presentation quality, testing reliability, or documentation depth.

10 Limitations and Future Improvements

10.1 Current Limitations

Even though Dormatrix is already broad in scope, the current design still leaves room for improvement:

- it remains a local file-based system rather than a concurrent networked platform;
- text-file persistence can become harder to manage as data volume grows;
- the system is highly optimized for the Windows + capable terminal demo environment;

- advanced analytics, search, and reporting are limited;
- UI screenshots and formal UML diagrams still need to be inserted into the final printed version of the report.

10.2 Future Enhancements

A realistic future roadmap could include:

- migrating from flat files to a relational database;
- adding multi-user concurrency control;
- introducing a web or GUI interface while keeping the current CLI as an admin-friendly mode;
- integrating notifications for complaints, token expiration, or announcements;
- generating printable reports automatically from store, complaint, or room data;
- improving analytics for occupancy, complaint trends, or meal demand forecasting;
- strengthening security with salt-based hashing and richer permission policies.

11 Personal Reflection Placeholders

11.1 Procheta Silvie

Reserved for personal reflection. Suggested points: learning from the store subsystem, balance logic, financial record tracking, and presentation-slide preparation; observations about translating technical work into a clear presentation narrative.

11.2 Khadiza Sultana

Reserved for personal reflection. Suggested points: lessons from authentication, UI design, cafeteria workflow, testing strategy, and report preparation; what improved most during implementation; what would be refined in a future version.

11.3 Sayma Tasnim

Reserved for personal reflection. Suggested points: development experience in the facility and lost & found modules; reflections on translating hall-life problems into booking logic and item-tracking workflows.

11.4 Ayman Binta Altaf Nondiny

Reserved for personal reflection. Suggested points: individual learning from complaint, announcement, and scheduling implementation; challenges faced in integrating multi-role workflows; experience contributing to testing and UI refinement.

12 Conclusion

Dormatrix succeeds as a comprehensive SPL project because it is more than a collection of separate CRUD screens. It is a coherent dormitory operating system built around user roles, operational workflows, and presentation quality. The codebase demonstrates substantial effort in design, implementation, and testing. It brings together technical decisions such as MVC + Repository structure, file persistence, custom libraries, password hashing, first-fit allocation, and policy-driven complaint escalation with strong user-facing decisions such as themed dashboards, keyboard navigation, and demo-time acceleration.

Most importantly, the project is grounded in a real problem environment. Every major feature responds to a dormitory pain point: room visibility, complaint delays, facility misuse, store dues, meal flexibility, announcements, scheduling, and item recovery. That connection between domain need and technical implementation is what makes Dormatrix a meaningful and well-justified final project.